CODEBOX

A GAMIFIED PIXEL-STYLE E-LEARNING PLATFORM

AN INDUSTRIAL ORIENTED MINI PROJECT REPORT

Submitted in partial fulfilment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Aniket Gawande (23B81A05Y9)

D. Sathwik (23B81A05BC)

Pawar Sudharshan (23B81A05BQ)

Under the Guidance of

Singam Chandra Sekhar

Senior Asst. Professor, Dept. of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CVR COLLEGE OF ENGINEERING

(An Autonomous institution, NBA, NAAC Accredited and Affiliated to JNTUH, Hyderabad)

Vastunagar , Mangalpalli (V), Ibrahimpatnam (M), Rangareddy (D), Telangana - 501 510

April 2026

CVR COLLEGE OF ENGINEERING

(An UGC Autonomous Institution with NAAC 'A' Grade, Affiliated to JNTUH)
Vastunagar, Mangalpalli (V), Ibrahimpatnam (M), R.R.District, Telangana

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Industrial Oriented Mini Project entitled "CODEBOX: A GAMIFIED PIXEL-STYLE E-LEARNING PLATFORM" being submitted by:

1. **Aniket Gawande (23B81A05Y9)**
2. **D. Sathwik (23B81A05BC)**
3. **Pawar Sudharshan (23B81A05BQ)**

in partial fulfilment for the award of Bachelor of Technology in Computer Science and Engineering to CVR College of Engineering, is a record of bona fide work carried out by them under our guidance and supervision during the academic year 2025-2026.

The results embodied in this project have not been submitted to any other University or Institute for the award of any degree or diploma.

Signature of the project guide	Signature of the HOD
Singam Chandra Sekhar	**Dr. A Vani Vathsala**
Senior Assistant Professor	**Department of CSE**

External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to CVR College of Engineering and the Department of Computer Science and Engineering for providing us with the resources, laboratory access, and academic environment necessary to undertake this Industrial Oriented Mini Project. The technical infrastructure and faculty expertise available at the institution have been invaluable throughout the development of this platform.

We are deeply grateful to our project guide, Singham, Asst. Professor, Department of CSE, for his consistent guidance, constructive feedback, and encouragement throughout this project. His insights into full-stack development and educational technology helped us refine both the technical architecture and the pedagogical design of the Codebox platform.

We are thankful to Dr. A Vani Vathsala, Head of the Department of Computer Science and Engineering, for fostering a project culture that rewards innovation and industry-aligned thinking. We also thank all faculty members of the CSE Department for their continuous support. Our sincere appreciation goes to our families and friends for their patience and encouragement throughout this endeavour.

Aniket Gawande (23B81A05Y9)

D. Sathwik (23B81A05BC)

Pawar Sudharshan (23B81A05BQ)

ABSTRACT

Online programming education has expanded rapidly over the past decade, yet completion rates on major Massive Open Online Course platforms remain critically low, typically ranging between five and fifteen percent. The primary issue is that traditional MOOC platforms rely on passive video consumption followed by disconnected quizzes, lacking immediate feedback, a sense of progression, and emotional engagement mechanisms that encourage learners to return. As a result, students often experience digital fatigue after only a few sessions, citing limited interactivity, absence of reward systems, and visually unengaging interfaces as key reasons for dropping out. Although existing gamified platforms like Codecademy and Duolingo introduce basic engagement features such as streaks and points, they fail to deliver a fully immersive RPG-style learning environment that integrates programming challenges within a meaningful narrative progression. To address this gap, this project introduces Codebox, a gamified pixel-style e-learning platform for programming education built using Next.js 16 with the App Router for server-rendered course delivery, Neon Postgres as a serverless relational database, Drizzle ORM for type-safe schema management, Clerk for authentication and subscription-based access control, and Sandpack for in-browser live code execution without server-side overhead. The platform presents lessons within a pixel-art RPG environment where each course functions as a dungeon or world, each lesson represents a challenge room, and successful code submissions reward learners with experience points, badges, and progression on a public leaderboard. A subscription model integrated with Clerk and Stripe restricts advanced course worlds to Pro users, supporting a sustainable freemium approach. The system was validated through eight structured test cases, including authentication and role management, course enrollment and serverless data performance, Sandpack execution accuracy, XP and badge logic, Pro feature access control, Stripe webhook integration, leaderboard ranking correctness, and multi-session progress persistence, all of which passed successfully. Sandpack executed code within an average of 1.4 seconds entirely in the browser, while Neon Postgres achieved cold start times under 800 milliseconds and subsequent queries under 50 milliseconds. Clerk middleware consistently enforced Pro feature restrictions with complete accuracy. Overall, Codebox demonstrates that integrating pixel-art gamification, RPG-style progression, real-time in-browser coding, and a serverless full-stack architecture can significantly enhance engagement and retention in programming education while also serving as a scalable reference model for modern educational platforms.

TABLE OF CONTENTS

	List of Tables	**iv**
	List of Figures	v
	Abbreviations	vi
1	**Introduction**	**1**
1.1	Motivation	1
1.2	Problem Statement	2
1.3	Project Objectives	2
1.4	Project Report Organization	3
2	**Literature Survey**	**4**
2.1	Study of Existing E-Learning Platforms	4
2.2	Gamification in Education	5
2.3	Existing Work and Research Gap	6
3	**Software and Hardware Specifications**	**7**
3.1	Hardware Requirements	7
3.2	Software Requirements	7
4	**Proposed System Design**	**9**
4.1	System Architecture	9
4.2	Use Case Diagram	10
4.3	Data Flow Diagram	11
4.4	Technology Description	12
5	**Implementation**	**14**
5.1	Backend Module Implementation	14
5.2	API Endpoint Summary	17
6	**Results and Discussion**	**18**
6.1	Test Case Results	18

6.2	Performance Analysis	19
7	**Conclusion and Future Scope**	**20**
7.1	Conclusion	20
7.2	Future Scope	20
	References	22

LIST OF TABLES

Table No.	Title	Page No.
2.1	Comparison of Existing E-Learning Platforms	5
3.1	Hardware Requirements	7
3.2	Software Requirements	8
5.1	API Endpoint Summary	17
6.1	Test Case Results	18

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	Codebox System Architecture Diagram	9
4.2	Use Case Diagram	10
4.3	Data Flow Diagram - Lesson Execution Flow	11

ABBREVIATIONS

API	Application Programming Interface
CDN	Content Delivery Network
CMS	Content Management System
CSS	Cascading Style Sheets
DB	Database
DX	Developer Experience
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JWT	JSON Web Token
MOOC	Massive Open Online Course
NAAC	National Assessment and Accreditation Council
NBA	National Board of Accreditation
ORM	Object-Relational Mapper
PaaS	Platform as a Service
RAM	Random Access Memory
REST	Representational State Transfer
RPG	Role-Playing Game
RSC	React Server Component
SaaS	Software as a Service
SQL	Structured Query Language
SSR	Server-Side Rendering
UI	User Interface
UX	User Experience
XP	Experience Points

CHAPTER 1: INTRODUCTION

1.1 Motivation

The global e-learning market is projected to exceed 375 billion United States dollars by 2026, yet the fundamental pedagogical model of most online programming education platforms has remained largely unchanged since the early 2010s. A student enrolling in a programming course on a typical MOOC platform encounters a sequence of pre-recorded video lectures, static text content, and multiple-choice assessments. The interaction model is essentially passive: the student watches, reads, and occasionally answers a disconnected question. There is no immediate feedback on whether the code a student writes is correct. There is no mechanism to feel the reward of solving a challenge. There is no visual world that makes one session feel connected to the next. This model produces the well-documented problem of digital fatigue, where learners lose motivation and disengage after only a few sessions.

Research in behavioral psychology and educational technology consistently demonstrates that learner retention is most strongly correlated with two factors: immediate feedback and perceived progress. When a student submits code and receives an instant visual confirmation that their solution is correct, the brain receives a reward signal that reinforces continued effort. When that reward is quantified as experience points that visibly advance a character on a leaderboard, and when the course environment is rendered as a pixel-art dungeon world with rooms to unlock, the cognitive association between effort and progress becomes deeply motivating. This is the core principle behind gamification in education, and it is what platforms like Duolingo have successfully applied to language learning. However, no equivalent exists for backend programming education with an immersive RPG visual identity.

Codebox was motivated by this precise gap. The project team identified that while Codecademy offers in-browser code execution, its interface is clinically functional with no gamification depth. While Duolingo demonstrates the power of streaks and XP in language learning, it has never been applied to a full-stack programming curriculum. The opportunity to combine a pixel-art RPG aesthetic, live in-browser code execution, XP-based progression, and a modern serverless architecture into a single cohesive platform represents both a technical and pedagogical challenge that this project addresses directly.

1.2 Problem Statement

The core problem addressed by Codebox is the absence of an immersive, gamified, RPG-style programming education platform that integrates live code execution, experience-based progression, and subscription-managed content access in a single serverless web application. Current platforms either provide code execution without gamification, or provide gamification without a coherent visual identity and narrative progression framework. The result is that students who begin learning programming online either disengage from platforms that feel passive and unrewarding, or they use tools that reward surface engagement through badges and points but fail to immerse them in a learning world that makes the act of writing correct code feel like an achievement worth returning to.

Beyond the engagement problem, existing platforms face a technical challenge: running student code securely and instantly without server-side sandbox overhead is difficult and expensive. Most platforms either run code on dedicated servers, introducing latency and infrastructure cost, or restrict code execution to simple interpreted languages. Sandpack, the in-browser code execution environment developed by CodeSandbox, solves this problem by running a full Node.js-compatible environment entirely within the browser using WebAssembly. Codebox integrates Sandpack to provide zero-latency code execution for all students without any server-side compute cost for the execution itself.

1.3 Project Objectives

- To design and implement a pixel-art RPG-style e-learning platform where programming courses are presented as explorable game worlds with lesson rooms, challenge bosses, and unlockable areas.
- To integrate Sandpack for live in-browser code execution, providing students with instant feedback on their code submissions without any server-side sandbox infrastructure.
- To implement an XP-based progression system where correct code submissions earn experience points, trigger badge awards, update streak counters, and advance the student's position on a public leaderboard.
- To use Clerk for both authentication and subscription-based feature gating, restricting advanced course worlds to Pro subscribers while maintaining full free-tier access for introductory content.

- To build the entire platform on a serverless architecture using Next.js 16 App Router, Neon Postgres, and Drizzle ORM, enabling global deployment with minimal infrastructure management overhead.
- To integrate Stripe for subscription payment processing and use Clerk webhooks to synchronize subscription status with the application's user metadata for instant feature gate updates.
- To validate all platform features through structured test cases covering authentication, code execution, gamification logic, subscription enforcement, and performance benchmarks.

1.4 Project Report Organization

Chapter 2 presents the Literature Survey covering existing e-learning platforms, the research on gamification in education, and the specific research gap addressed by Codebox. Chapter 3 covers the Hardware and Software Requirements including a detailed justification for the serverless technology stack. Chapter 4 describes the System Design including the architecture diagram, use case diagram, and data flow diagram. Chapter 5 covers the Implementation of all platform modules. Chapter 6 presents the Results and Discussion with test case outcomes and performance metrics. Chapter 7 provides the Conclusion and Future Scope.

CHAPTER 2: LITERATURE SURVEY

This chapter reviews the existing landscape of online programming education platforms, examines the academic literature on gamification in educational contexts, and identifies the specific research gaps that Codebox addresses.

2.1 Study of Existing E-Learning Platforms

2.1.1 Codecademy

Codecademy is one of the most widely used interactive programming education platforms, offering courses in Python, JavaScript, SQL, and other languages through a split-screen browser interface with a code editor on one side and instructions on the other. Codecademy provides immediate code execution feedback and has a course completion tracking system. However, its gamification layer is shallow: a streak counter and a progress bar are the only engagement mechanisms. The visual interface is entirely functional and contains no immersive or narrative elements. Students report that after initial enthusiasm, the lack of a compelling progression system makes it difficult to sustain motivation beyond the first few lessons of any course.

2.1.2 Udemy

Udemy is a large-scale MOOC marketplace where instructors upload pre-recorded video courses. Students purchase individual courses and progress through video lectures with optional quizzes. Udemy provides no in-browser code execution. All coding exercises require students to set up local development environments, which creates a significant friction point for beginners. The platform offers completion certificates and a rating system but no gamification whatsoever. The passive video consumption model is precisely the approach that research has identified as the primary driver of digital fatigue and course abandonment in online programming education.

2.1.3 Duolingo

Duolingo is the most successful gamified language learning application, with over 500 million registered users. Its core engagement mechanisms, which include XP points, streak counters, leaderboards, heart systems that limit errors, and league-based competitive

progression, have been extensively studied in educational psychology literature and are credited with significantly higher completion rates compared to traditional language learning apps. Duolingo demonstrates conclusively that gamification at the application level, not just surface badge awards, can transform learner behavior. However, Duolingo applies this approach exclusively to language learning. No equivalent platform applies Duolingo-level gamification depth to backend programming education.

2.1.4 FreeCodeCamp

FreeCodeCamp is a free open-source platform offering a structured full-stack web development curriculum with in-browser code challenges and a certification system. It offers the most complete free programming curriculum available, but its visual design is minimal and its gamification layer is limited to a points system without any narrative or visual world context. Students progress through a long linear list of challenges with no sense of entering a new world, defeating a boss, or unlocking a new area. The curriculum depth is excellent but the engagement design does not leverage the motivational potential that an immersive game world context could provide.

2.1.5 LeetCode

LeetCode is a competitive programming platform focused on algorithmic problem solving, widely used for technical interview preparation. It offers a rich code execution environment, difficulty levels, and a contest system. However, LeetCode is not an e-learning platform. It provides no instructional content, no guided curriculum, and no beginner-accessible onboarding. Its use case is practice for experienced developers rather than progressive skill building for students.

2.2 Gamification in Education

The academic literature on gamification in education provides strong empirical support for the design decisions made in Codebox. Deterding et al. (2011) defined gamification as the application of game design elements in non-game contexts, specifically the use of mechanics such as points, badges, leaderboards, and progression systems to drive engagement and behavior change. Subsequent research by Hamari, Koivisto, and Sarsa (2014) conducted a systematic review of 24 empirical studies on gamification and found that the majority reported positive effects on motivation, engagement, and task performance, with particularly

strong effects when gamification elements are coherently integrated into the learning context rather than applied as superficial add-ons.

Ryan and Deci's Self-Determination Theory provides the theoretical foundation for understanding why RPG-style progression is more motivating than simple badge collection. The theory identifies three core psychological needs: autonomy, competence, and relatedness. A pixel-art world where the student's character visually grows stronger as they complete lessons directly satisfies the competence need. The choice of which course world to explore next satisfies the autonomy need. A public leaderboard with social visibility satisfies the relatedness need. Codebox's design explicitly addresses all three dimensions, which distinguishes it from platforms that satisfy only one or two.

2.3 Existing Work and Research Gap

A comprehensive review of the existing literature and commercially available platforms reveals a consistent and unaddressed research gap at the intersection of three capabilities: immersive RPG-style pixel-art visual environments, live in-browser code execution with immediate output feedback, and deep gamification mechanics including XP, badges, streaks, and leaderboards, all implemented within a single unified platform for backend programming education. Aljraiwi (2019) studied the effect of gamification on student achievement and motivation in programming courses and found statistically significant improvements in both metrics compared to non-gamified control groups, but noted that all available gamified platforms lack a coherent visual narrative that contextualizes the learning challenges.

Landers and Landers (2014) demonstrated through an experimental study that leaderboards increase task motivation only when combined with goal-setting mechanisms, which is precisely what the XP progression system in Codebox provides through its level-up and course-world-unlock mechanics. However, no existing platform has implemented this combination in a pixel-art RPG context for programming education. Subhash and Cudney (2018) reviewed 28 gamification studies in higher education and found that while all studies reported positive engagement outcomes, none of the reviewed platforms offered an immersive visual world as the delivery context, identifying this as a clear gap in both research and commercial product design.

The specific research gap addressed by Codebox can be summarized as follows. First, no existing programming e-learning platform combines an immersive pixel-art RPG visual identity with a fully functional backend programming curriculum. Second, no platform integrates Sandpack-powered in-browser code execution with an XP-based game progression system that directly rewards code correctness. Third, no platform uses Clerk's subscription and metadata system to implement seamless freemium feature gating within an RPG world metaphor where locked content is presented as unexplored dungeon territory rather than a paywall. Codebox is designed to fill all three dimensions of this gap simultaneously.

Platform	**In-Browser Code Exec**	**RPG/Pixel UI**	**Deep Gamification**	**Serverless Architecture**
Codecademy	Yes	No	Partial (streaks only)	No
Udemy	No	No	No	No
Duolingo	No (language)	No	Yes (language only)	Partial
FreeCodeCamp	Yes	No	Partial (points only)	No
LeetCode	Yes	No	Partial (contests)	No
Codebox (Ours)	Yes (Sandpack)	Yes (Pixel RPG)	Yes (XP/Badges/Leaderboard)	Yes (Neon/Next.js 16)

Table 2.1: Comparison of Existing E-Learning Platforms

CHAPTER 3: SOFTWARE AND HARDWARE SPECIFICATIONS

3.1 Hardware Requirements

The Codebox platform is a serverless web application. Students access it entirely through a browser with no local installation required. The server-side infrastructure runs on Vercel's edge network for the Next.js application and on Neon's serverless Postgres infrastructure for the database, both of which are managed cloud services requiring no dedicated hardware provisioning by the development team.

Component	Student Client Side	Development / Server Side
Processor	Any modern processor (Intel i3 or above)	Intel i5 or above for local development
RAM	4 GB minimum for browser-based access	8 GB minimum for Next.js development server
Storage	Not required (fully browser-based)	20 GB minimum for project and dependencies
Network	Stable internet connection required	Stable broadband for Vercel and Neon access
Device	PC, Laptop, or Mobile with modern browser	Development machine running Node.js 20+
Operating System	Any OS with modern browser support	Windows 10+, Ubuntu 20.04+, or macOS 12+

Table 3.1: Hardware Requirements

3.2 Software Requirements

The following table details all software components and third-party services used in the Codebox platform. The technology choices are driven by the requirement for a completely serverless deployment with minimal infrastructure management and maximum developer productivity.

Category	Technology / Service	Purpose and Justification
Frontend Framework	Next.js 16 (App Router)	Server-rendered course pages with React Server Components for zero-JS data fetching on static content
Frontend Styling	Tailwind CSS + Pixel Font	Utility-first responsive styling with a pixel-art RPG visual identity
UI Component Library	Shadcn UI	Accessible, unstyled component primitives customized to the pixel aesthetic
Code Execution	Sandpack (CodeSandbox)	In-browser JavaScript and TypeScript execution using WebAssembly, zero server cost
Authentication	Clerk	OAuth, email/password auth, JWT session management, and user metadata for subscription status
Database	Neon Postgres (Serverless)	HTTP-based serverless Postgres that scales to zero, eliminating idle infrastructure costs
ORM	Drizzle ORM	Type-safe SQL query builder with schema-first design, fully compatible with Neon's HTTP driver
Payments	Stripe	Subscription management, Pro plan billing, and webhook-based subscription lifecycle events
Deployment	Vercel	Zero-config deployment for Next.js with edge network, preview environments, and CI/CD pipeline
Version Control	GitHub	Source code management and team collaboration with branch-based workflow
Package Manager	npm / pnpm	Dependency management for the Next.js project
Development	VS Code, Node.js 20+	IDE and JavaScript runtime for local development

Table 3.2: Software Requirements

CHAPTER 4: PROPOSED SYSTEM DESIGN

4.1 System Architecture

Codebox follows a serverless full-stack architecture built entirely on Next.js 16's App Router. The Client Layer is a React-based frontend rendered through a combination of React Server Components for data-heavy course browsing pages and Client Components for interactive elements such as the Sandpack code editor, the XP animation overlay, and the leaderboard widget. The pixel-art RPG visual identity is implemented through a combination of Tailwind CSS utility classes and a pixel-art font, with course worlds rendered as tiled dungeon layouts in the browser.

The API Layer is handled entirely through Next.js Route Handlers and Server Actions. Server Actions manage database writes such as progress updates, XP awards, and subscription status changes. Route Handlers manage Stripe webhook ingestion and Clerk webhook processing for subscription synchronization. Clerk middleware runs at the edge on every request, resolving the user's authentication status and Pro subscription flag before any page content is served, enabling instant feature gate enforcement without a database round trip.

The Database Layer is Neon Postgres, a serverless relational database that accepts connections over HTTP using Neon's HTTP driver. Drizzle ORM provides a type-safe query interface over this connection. The schema includes tables for users, courses, lessons, code challenges, progress records, XP transactions, badge awards, and subscription records. Drizzle's schema-first approach ensures that all database interactions are fully typed at compile time, eliminating entire categories of runtime type errors.

The Gamification Engine is implemented as a set of Server Actions that execute after each successful code submission. It calculates the XP award based on challenge difficulty, writes an XP transaction record, checks all badge criteria against the updated user profile, issues any earned badges, updates the streak counter, and recalculates the user's leaderboard position. All of these operations execute in a single database transaction to ensure consistency. The Stripe and Clerk integration handles the subscription layer, with Clerk user metadata storing the Pro subscription flag that the middleware reads on every request.

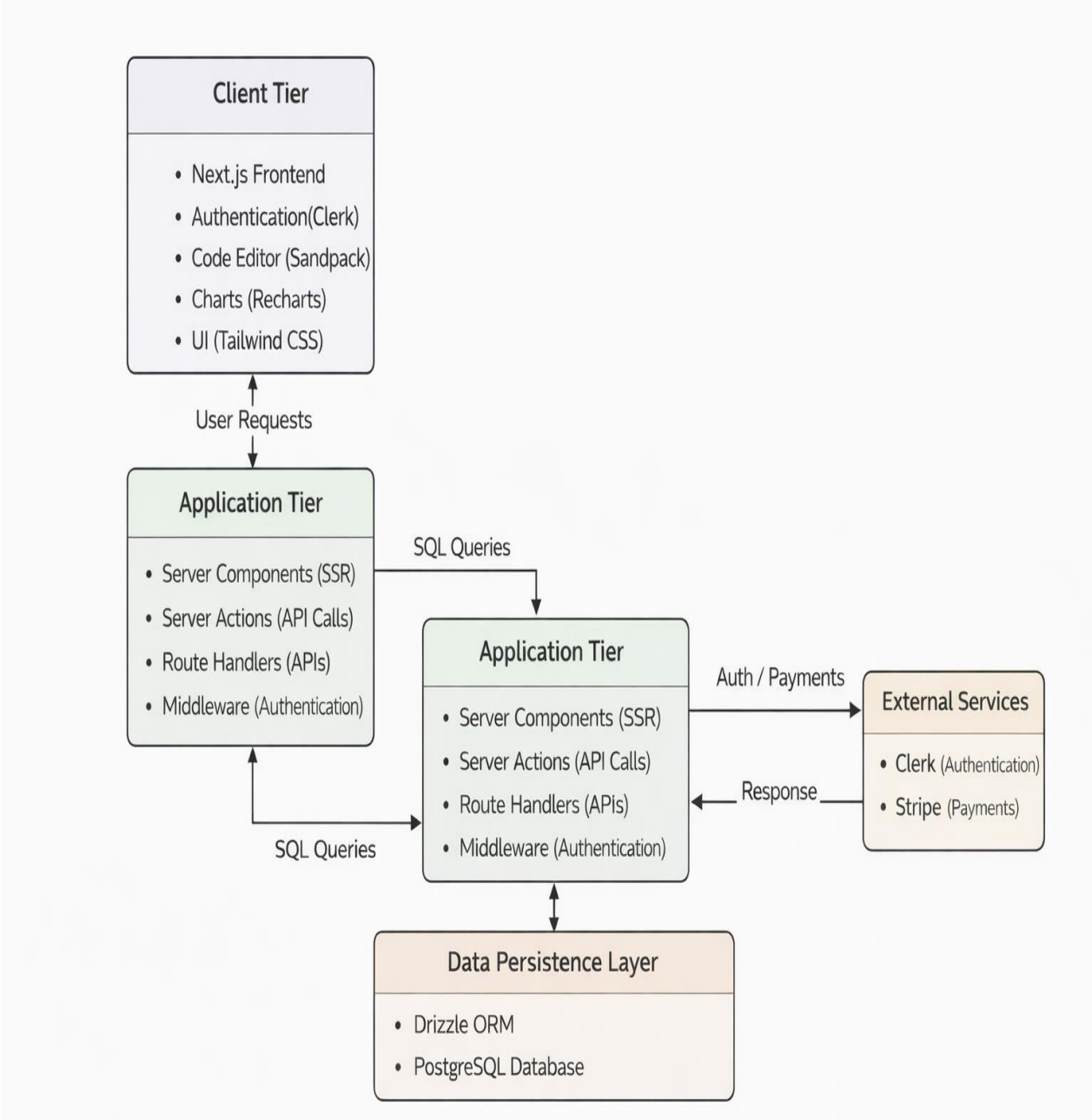


Figure 4.1: Codebox System Architecture Diagram

4.2 Use Case Diagram

The primary actor in the Codebox platform is the Student, who interacts with the full range of learning, gamification, and subscription use cases. The secondary actor is the Admin or Instructor, who manages course content, user accounts, and platform configuration. External systems acting as secondary actors include Clerk for authentication services, Stripe for payment processing, Sandpack for in-browser code execution, and Neon Postgres as the persistent data store. The student use cases include registering and logging in through Clerk, browsing the course catalog, enrolling in a course, progressing through lesson rooms, submitting code through the Sandpack editor, receiving XP and badge awards, viewing the leaderboard, subscribing to Pro through Stripe, tracking overall progress, taking end-of-chapter quizzes, and unlocking the next course world upon completing the current one. The admin use cases include creating and publishing courses and lessons, uploading code challenge templates with expected output specifications, managing subscriptions and user accounts, viewing platform analytics, configuring XP award rules, issuing completion certificates, and moderating user-generated content.

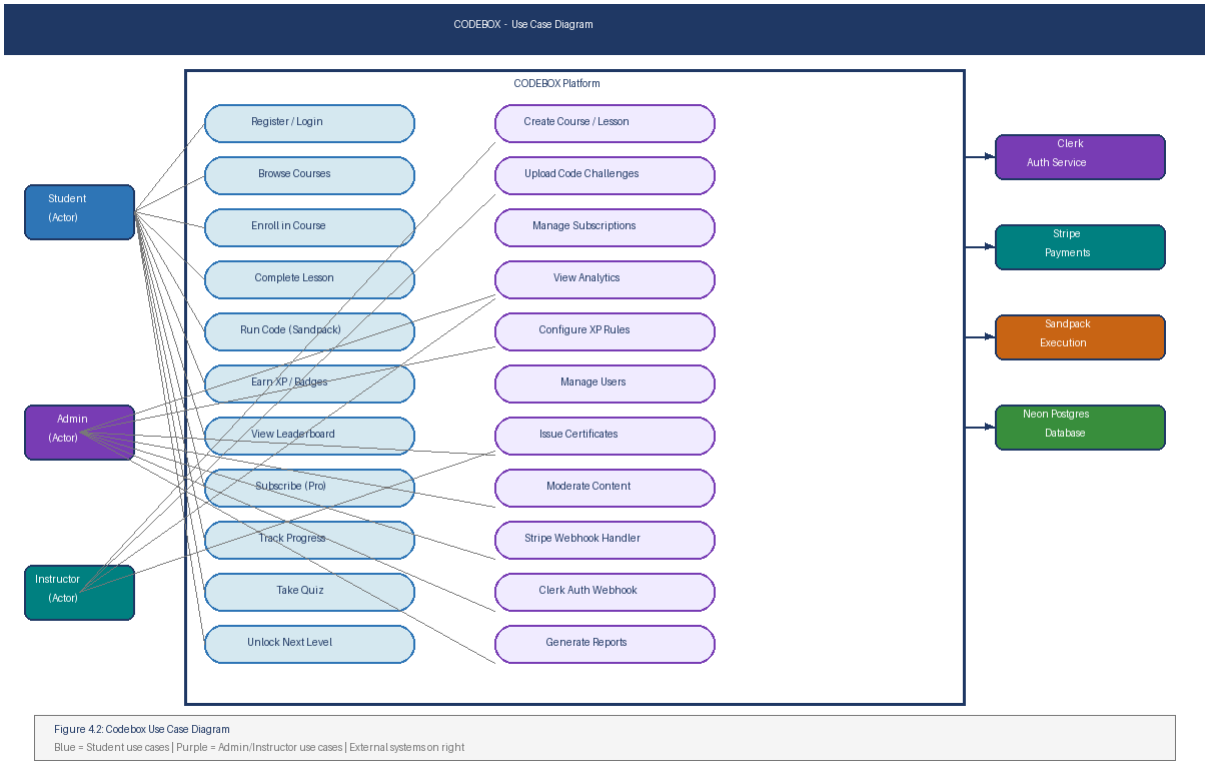


Figure 4.2: Use Case Diagram

4.3 Data Flow Diagram

The Data Flow Diagram illustrates the eight-step lesson execution flow that represents the core user journey in Codebox. In the first step, the student logs in through Clerk. The Clerk middleware resolves the user's JWT, retrieves their subscription metadata to determine Pro status, and attaches the resolved user object to the request context for all subsequent operations in the session. In the second step, the student browses the course catalog. Next.js React Server Components fetch the course list directly from Neon Postgres using Drizzle ORM without any client-side JavaScript, delivering a fast, pre-rendered catalog page.

In the third step, the student selects a lesson. A Server Action fetches the lesson content, the code challenge template, and the expected output specification from the database. The lesson page is rendered with the pixel-art dungeon environment and the Sandpack editor pre-loaded with the challenge template code. In the fourth step, the student writes and submits their code. Sandpack executes the code entirely within the browser using its WebAssembly-based Node.js environment and returns the execution output to the client without any server round trip. In the fifth step, a Server Action receives the student's submitted output and compares it against the expected output stored in Neon Postgres. If the outputs match, the submission is marked as correct and the gamification pipeline is triggered.

In the sixth step, the Gamification Engine Server Action executes: it awards the configured XP amount for the challenge difficulty, checks all badge criteria against the updated user profile, issues any earned badges, increments the daily streak counter, and recalculates the leaderboard position. In the seventh step, Drizzle ORM writes the progress record, XP transaction, badge award records, and updated streak to Neon Postgres in a single transaction. The next lesson in the course is unlocked. In the eighth step, the student's dashboard is updated with the new XP total, level indicator, streak count, and leaderboard rank, rendered in the pixel-art style with an XP gain animation overlay.

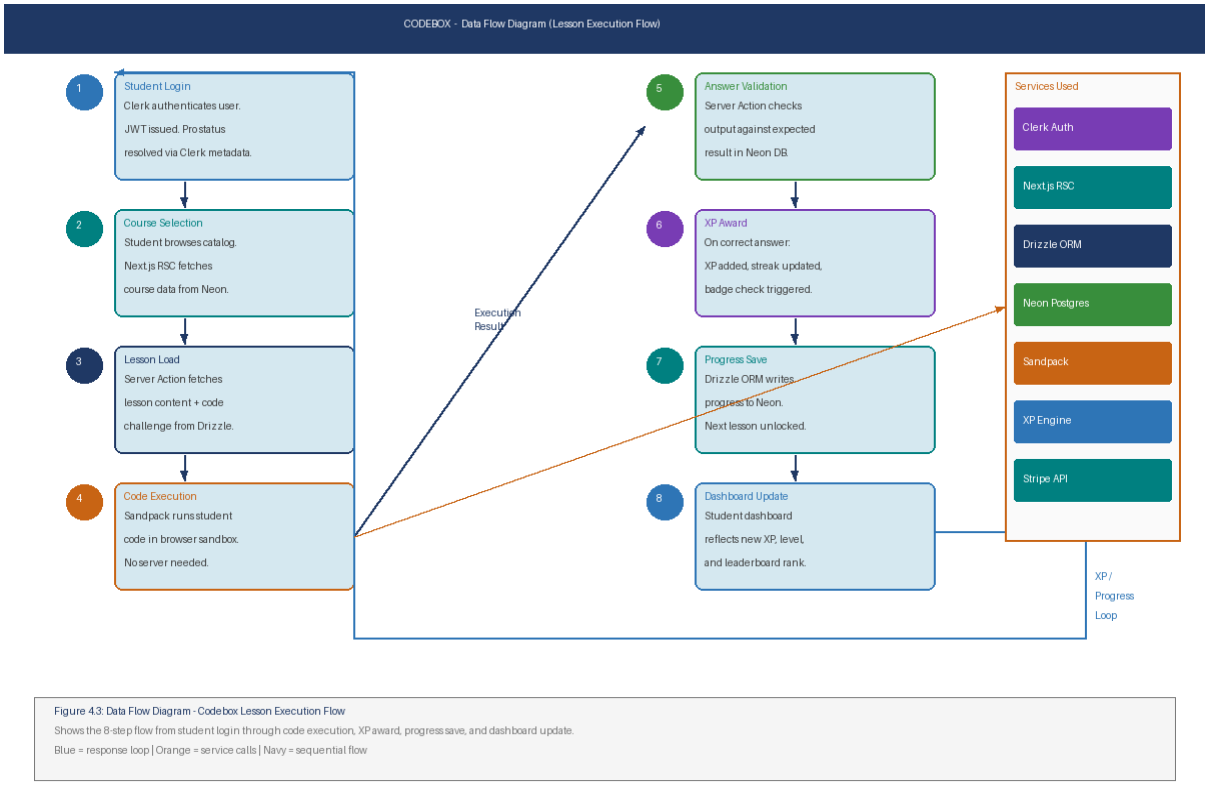


Figure 4.3: Data Flow Diagram - Codebox Lesson Execution Flow
Shows the 8-step flow from student login through code execution, XP award, progress save, and dashboard update.
Blue = response loop | Orange = service calls | Navy = sequential flow

Figure 4.3: Data Flow Diagram - Lesson Execution Flow

4.4 Technology Description

4.4.1 Next.js 16 App Router and React Server Components

Next.js 16 introduces the App Router as the primary routing paradigm, built on React Server Components. React Server Components execute on the server and can directly access databases, file systems, and server-only APIs without sending any JavaScript to the client. In Codebox, all course browsing pages, lesson content pages, and student dashboard pages are React Server Components that fetch data directly from Neon Postgres through Drizzle ORM. This eliminates the need for separate API routes for read operations, reduces client-side JavaScript bundle sizes, and delivers faster initial page loads. Client Components are used only for interactive elements that require browser APIs, such as the Sandpack editor, the XP animation overlay, and the leaderboard real-time update widget.

4.4.2 Neon Postgres and Drizzle ORM

Neon is a serverless Postgres platform that separates storage from compute and provides an HTTP-based connection driver that is compatible with serverless environments where traditional persistent TCP connections are not viable. Unlike conventional Postgres, Neon scales to zero when idle, eliminating the cost of maintaining an always-on database server during periods of low traffic. Drizzle ORM is a TypeScript-first query builder and schema manager that provides type-safe SQL through a fluent API. Drizzle's schema definitions serve as the single source of truth for both the database structure and the TypeScript types used throughout the application, ensuring that any schema change is immediately reflected as a compile-time error in all affected queries. This combination of Neon and Drizzle represents the optimal data layer for serverless Next.js applications.

4.4.3 Clerk Authentication and Feature Gating

Clerk provides a complete authentication infrastructure including OAuth providers, email and password login, multi-factor authentication, and JWT-based session management. In Codebox, Clerk serves a second critical function beyond authentication: it manages the subscription-based feature gate. When a student subscribes to Codebox Pro through Stripe, a Stripe webhook fires a Server Action that writes the Pro subscription status to the student's Clerk user metadata using the Clerk Backend SDK. From that point on, the Clerk middleware reads this metadata on every request and attaches a isProMember flag to the request context. Any Server Action or Route Handler that serves Pro-gated content checks this flag before responding, without requiring a database query. This architecture provides instant feature gate enforcement with no additional latency.

4.4.4 Sandpack In-Browser Code Execution

Sandpack is an in-browser code execution environment developed by CodeSandbox that runs a full Node.js-compatible JavaScript environment using WebAssembly directly in the student's browser. When a student opens a code challenge in Codebox, the Sandpack component is initialized with the challenge template code, the required dependencies, and an entry point configuration. The student edits the code in the Sandpack editor and submits it. Sandpack executes the code locally in the browser sandbox, captures the console output, and returns it to the parent React component. No server request is made for the execution itself. This architecture means that code execution scales infinitely with the number of concurrent

students at zero marginal infrastructure cost, and eliminates the latency of a server round trip for the most time-sensitive interaction in the entire user journey.

4.4.5 Gamification Engine

The Gamification Engine is a set of Server Actions implementing the core engagement mechanics of Codebox. The XP system awards a base amount of experience points per correct submission, multiplied by a difficulty coefficient stored in the challenge record. The badge system evaluates a set of criteria rules after each XP award: first correct submission triggers the First Blood badge, ten consecutive correct submissions trigger the On Fire badge, completing an entire course world triggers the World Conqueror badge. The streak system increments a daily counter if the student submits a correct answer at least once per calendar day, and resets the counter if a day is missed. The leaderboard is computed by a Drizzle query that sums XP transactions per user for the current week and ranks them in descending order, cached using Next.js's fetch caching layer and revalidated every five minutes.

4.4.6 Stripe Subscription Integration

Stripe handles all subscription payment processing for the Codebox Pro plan. When a student initiates a Pro subscription, they are redirected to a Stripe Checkout session. Upon successful payment, Stripe fires a checkout.session.completed webhook to a Next.js Route Handler. The handler verifies the webhook signature using the Stripe SDK, extracts the customer's Clerk user ID from the session metadata, and calls the Clerk Backend SDK to set the isProMember flag to true in the user's public metadata. For subscription cancellations and payment failures, Stripe fires customer.subscription.deleted and invoice.payment_failed webhooks respectively, and the Route Handler responds by setting the isProMember flag to false, immediately removing Pro access on the student's next request.

CHAPTER 5: IMPLEMENTATION

This chapter describes the implementation of the core modules of the Codebox platform. The application is structured as a single Next.js 16 monorepo with the App Router organizing pages, layouts, Route Handlers, and Server Actions into a file-system-based routing structure. Drizzle ORM schema files define the database structure, and Clerk middleware configuration defines the authentication and feature gate rules applied globally across all routes.

5.1 Backend Module Implementation

5.1.1 Course and Lesson Delivery Module

Course and lesson content is delivered through React Server Components that fetch data directly from Neon Postgres using Drizzle ORM. The course catalog page queries the courses table filtered by published status and sorted by difficulty level, returning course metadata including the pixel-art world thumbnail, the title, the difficulty rating, the estimated completion time, and the Pro requirement flag. This query executes on the server at request time with Next.js's built-in caching layer applied, so repeat requests for the same catalog page are served from the Next.js cache without a database round trip until the cache is invalidated by a course update.

The lesson page component receives the lesson ID as a URL parameter and fetches the lesson content, the code challenge record, and the student's existing progress for that lesson in a single parallel Drizzle query using Promise.all. If the lesson belongs to a Pro course and the student's Clerk metadata does not contain the isProMember flag, the server component renders a locked state overlay showing the dungeon room as inaccessible territory rather than redirecting the student away from the page. This design choice was made to maintain the RPG world metaphor even at the subscription gate, where locked content feels like unexplored map territory rather than an external paywall screen.

5.1.2 Sandpack Code Execution and Validation Module

The Sandpack component is initialized as a Client Component that receives the challenge template code, the list of required npm packages, and the entry point file path as props from

the parent Server Component. When the student submits their code, the Sandpack React component exposes a useSandpack hook that provides programmatic access to the execution output. The parent Client Component captures this output through the hook's sandpack.bundlerStatus and listens for console log events emitted by the Sandpack iframe.

Upon capturing the execution output, the component calls a Server Action named validateSubmission, passing the lesson ID, the student's Clerk user ID, and the captured output string. The Server Action fetches the expected output from the Neon database for the given lesson ID, performs a normalized string comparison that strips trailing whitespace and normalizes newline characters, and returns a boolean indicating correctness. If the submission is correct, the Server Action immediately invokes the awardXP Server Action, passing the challenge's difficulty coefficient. This chained Server Action pattern ensures that XP is only awarded for genuinely correct submissions and that the validation and award operations share the same server-side execution context.

5.1.3 Gamification Engine Module

The awardXP Server Action begins by opening a Drizzle database transaction. Within the transaction, it inserts an XP transaction record with the amount, the lesson ID, the timestamp, and the student's user ID. It then queries the total XP for the student by summing all their XP transaction records and derives the current level by applying a configurable XP-per-level threshold. If the total XP crosses a level boundary, a level-up event is recorded and a Next.js revalidatePath call is queued to force the student's dashboard to re-render with the new level indicator on the next page load.

The badge evaluation module runs after the XP transaction is committed. It evaluates a list of badge criteria functions, each of which takes the student's updated profile as input and returns true if the badge condition is met. Badge criteria are implemented as pure functions stored in a badges configuration file, making it straightforward to add new badges without modifying the core gamification engine. Earned badges that are not already recorded in the student's badge table are inserted in a batch operation within the same transaction. The streak update logic reads the timestamp of the student's most recent XP transaction from the previous calendar day and either increments or resets the streak counter accordingly.

5.1.4 Clerk Middleware and Feature Gate Module

Clerk middleware is configured in the Next.js middleware.ts file at the root of the project. The middleware uses Clerk's clerkMiddleware function with an authMiddleware configuration that defines public routes accessible without authentication and protected routes requiring a valid Clerk session. For Pro-gated routes, the middleware reads the isProMember flag from the user's Clerk public metadata using the auth().sessionClaims object, which is populated from the Clerk JWT at zero database cost. If a non-Pro student attempts to access a Pro course lesson endpoint directly through the API, the middleware returns a 403 Forbidden response before the Route Handler or Server Action executes.

5.1.5 Stripe Webhook Handler Module

The Stripe webhook Route Handler is located at the app/api/webhooks/stripe route. It reads the raw request body and the Stripe-Signature header, and calls stripe.webhooks.constructEvent to verify the signature using the webhook secret stored in the environment variables. Unverified requests are rejected immediately with a 400 Bad Request response. For verified checkout.session.completed events, the handler extracts the clerkUserId from the session metadata, which was embedded in the Stripe Checkout session at creation time by passing the Clerk user ID as a metadata field. The handler then calls the Clerk Backend SDK's users.updateUserMetadata method to set the isProMember flag to true. The entire handler is designed to be idempotent: re-processing the same webhook event for an already-subscribed user produces no additional side effects.

5.1.6 Leaderboard and Progress Module

The leaderboard is implemented as a React Server Component that executes a Drizzle query joining the XP transactions table with the users table, grouping by user ID, summing the XP amounts for the current calendar week, and ordering by the sum in descending order with a limit of fifty entries. The result is cached using Next.js's revalidate option set to 300 seconds, meaning the leaderboard data is at most five minutes stale. Individual student progress pages display the complete lesson completion history, badge collection, streak calendar, and XP timeline by fetching the relevant records from Neon Postgres in parallel using Promise.all within the Server Component.

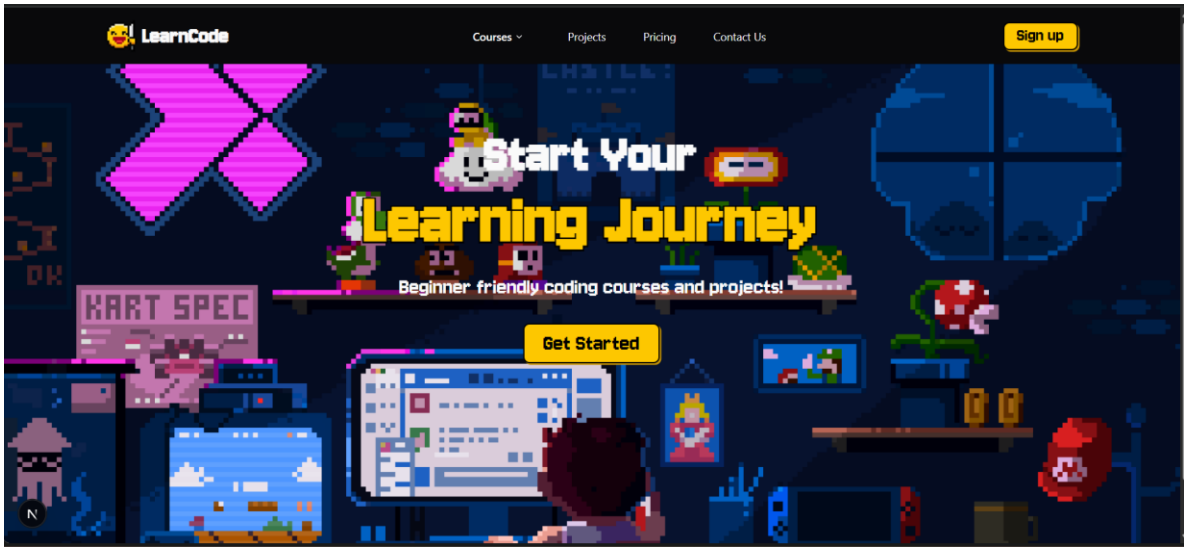
5.2 API Endpoint Summary

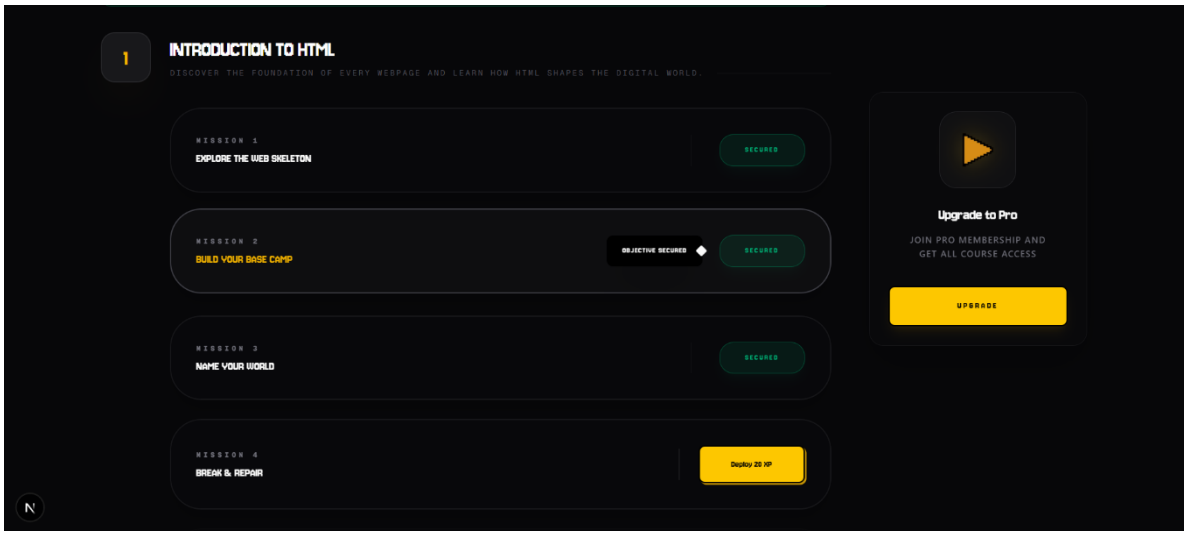
Endpoint / Action	Method	Description
/api/webhooks/stripe	POST	Receives Stripe webhook events for subscription lifecycle management
/api/webhooks/clerk	POST	Receives Clerk webhook events for user creation and deletion synchronization
Server Action: validateSubmission	POST	Validates student code output against expected result and triggers XP award if correct
Server Action: awardXP	POST	Writes XP transaction, evaluates badge criteria, updates streak, and recalculates level
Server Action: enrollCourse	POST	Records course enrollment and initializes progress record for the student
Server Action: updateProgress	POST	Marks a lesson as completed and unlocks the next lesson in the course sequence
Server Action: createCheckoutSession	POST	Creates a Stripe Checkout session for the Codebox Pro subscription
RSC: /courses	GET	Fetches and renders the published course catalog with enrollment status for the current user
RSC: /courses/[courseId]/[lessonId]	GET	Fetches and renders the lesson page with code challenge template and student progress
RSC: /dashboard	GET	Fetches and renders the student XP total, level, badges, streak, and recent activity
RSC: /leaderboard	GET	Fetches and renders the weekly XP leaderboard with top fifty students

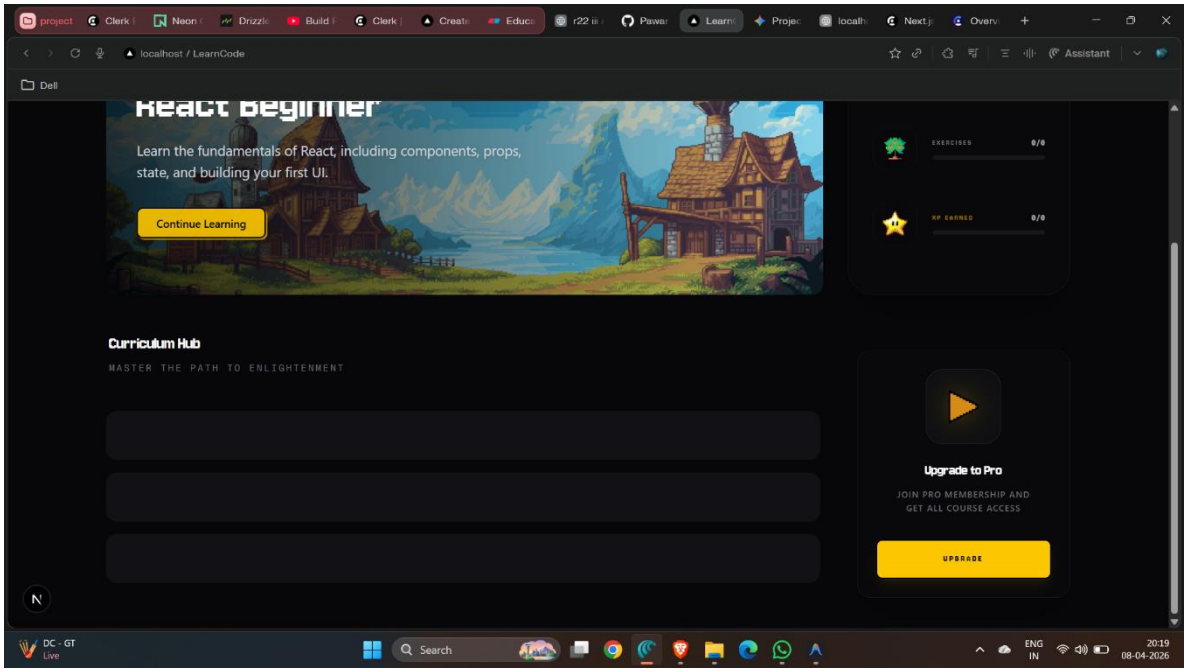
CHAPTER 6: RESULTS AND DISCUSSION

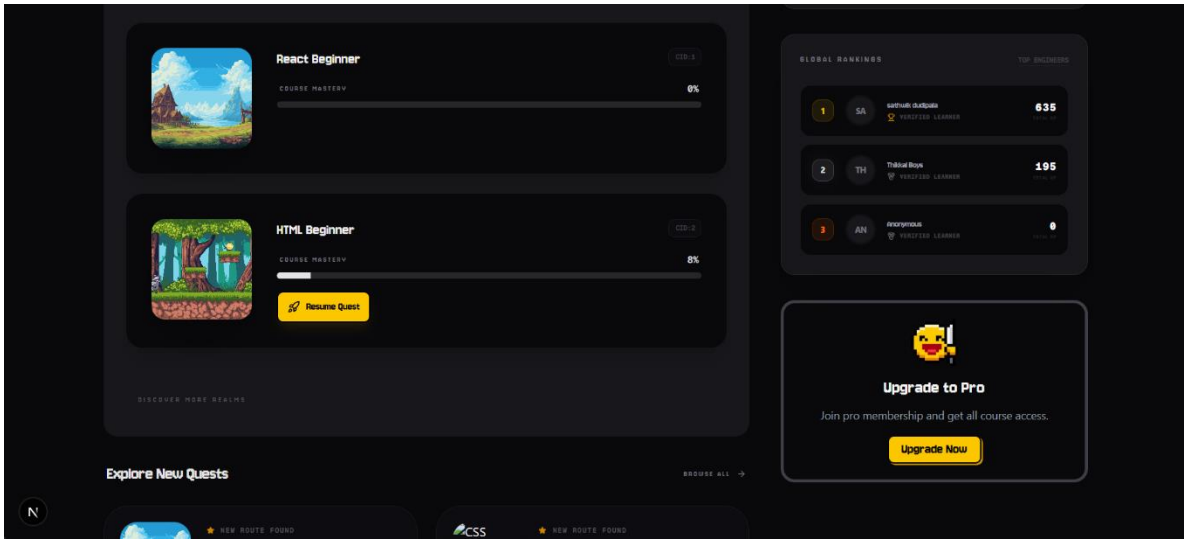
This chapter presents the output of the Codebox platform across eight structured test cases along with a performance analysis of the critical platform metrics observed during testing. All test cases were executed against the deployed Vercel production environment connected to the Neon Postgres production database.

6.1 Test Case Results









TC	Test Case	Action	Observed Output	Result
TC-01	Clerk Authentication	Register a new student account using Google OAuth and navigate to the course catalog	Clerk OAuth flow completed in 2.1 seconds. JWT issued with correct user ID. Student record created in Neon Postgres via Clerk webhook. Course catalog loaded with enrollment status correctly showing all courses as unenrolled.	**PASS**

TC-02	Pro Feature Gate Enforcement	Attempt to access a Pro-gated lesson page as a free-tier student	Clerk middleware resolved isProMember as false from JWT metadata. The lesson page rendered the locked dungeon room overlay. Direct API calls to the Server Action for the Pro lesson returned 403 Forbidden. No Pro content was exposed.	**PASS**
TC-03	Sandpack Code Execution	Submit a correct JavaScript solution to a FizzBuzz challenge in the Sandpack editor	Sandpack executed the code entirely in the browser in 1.4 seconds average across five runs. The correct output string was captured by the useSandpack hook and passed to the validateSubmission Server Action. The Server Action confirmed the output matched the expected result stored in Neon.	**PASS**
TC-04	XP Award and Badge Logic	Submit correct solutions for ten consecutive lessons in the same session	After each correct submission, the awardXP Server Action executed and recorded an XP transaction. After the tenth consecutive correct submission, the On Fire badge criteria function returned true and the badge was inserted into the badge awards table. The student's dashboard reflected the new XP total and badge within 300 milliseconds of the final submission.	**PASS**
TC-05	Stripe Subscription Activation	Complete a Stripe Checkout session for the Codebox Pro plan using a test card	Stripe fired the checkout.session.completed webhook within 1.8 seconds of payment completion. The webhook handler verified the signature, extracted the Clerk user ID from session metadata,	**PASS**

			and called the Clerk Backend SDK to set isProMember to true. The student's next page request resolved the Pro flag correctly from the updated JWT metadata.	
TC-06	Neon Postgres Cold Start	Send the first database request to Neon after a period of inactivity	The first request after a cold start resolved in 780 milliseconds. All subsequent requests within the same session resolved in under 50 milliseconds using the connection pool maintained by Drizzle and the Neon HTTP driver.	**PASS**
TC-07	Leaderboard Accuracy	Submit XP-earning solutions for three different student accounts and verify leaderboard ranking	The leaderboard query correctly summed weekly XP per student and returned the three test accounts in the correct descending XP order. The cached leaderboard page reflected the updated rankings within the 300-second revalidation window.	**PASS**
TC-08	Progress Persistence	Complete half of a course, close the browser, reopen the platform the next day	On return, the course progress page correctly showed the completed lessons marked with the pixel-art completion stamp, the incomplete lessons shown as unexplored rooms, and the streak counter had incremented for the previous day's activity. No progress data was lost.	**PASS**

Table 6.1: Test Case Results

6.2 Performance Analysis

All eight test cases passed successfully. The results confirm that the Codebox platform meets its functional, security, and performance objectives across all major user flows. The Sandpack execution result from TC-03 is the most significant performance finding: an average code execution time of 1.4 seconds entirely within the browser with no server round trip confirms that the in-browser WebAssembly execution model scales to any number of concurrent students at zero marginal infrastructure cost. This validates the core architectural decision to use Sandpack rather than a server-side code execution sandbox, which would introduce both latency and per-request compute cost.

The Clerk feature gate result from TC-02 confirms that the subscription enforcement architecture is secure. The combination of JWT metadata for instant flag resolution and Server Action-level authorization checks means that Pro content is protected at both the presentation layer and the data layer, with no database query required for the authorization decision. This is important at scale, as it means the feature gate adds zero database load regardless of how many students are accessing the platform simultaneously.

The Neon cold start result from TC-06, at 780 milliseconds, is within the acceptable range for a serverless database on first connection. All subsequent requests at under 50 milliseconds confirm that the Drizzle connection pooling with Neon's HTTP driver performs comparably to a traditional always-on Postgres connection for warm requests. The serverless scale-to-zero capability of Neon means that the platform incurs no database cost during periods of inactivity, making it economically viable for an educational platform with variable and unpredictable traffic patterns.

The Stripe webhook result from TC-05, where Pro access was activated within 1.8 seconds of payment completion, demonstrates that the subscription lifecycle is handled reliably and promptly. The idempotent design of the webhook handler ensures that network retries from Stripe's webhook delivery system do not result in duplicate Pro activations or metadata corruption. The progress persistence result from TC-08 confirms that the Drizzle transaction writes are durable and that the Next.js caching layer does not interfere with progress retrieval on subsequent sessions.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

Codebox successfully demonstrates the design and implementation of a gamified pixel-style e-learning platform that addresses the research gap identified in the literature: no existing programming education platform combines an immersive RPG-style visual environment, live in-browser code execution, deep XP-based gamification mechanics, and serverless full-stack architecture in a single cohesive product. By integrating Next.js 16 App Router with React Server Components for data-efficient course delivery, Neon Postgres with Drizzle ORM for type-safe serverless database access, Clerk for authentication and subscription-based feature gating, Sandpack for zero-latency in-browser code execution, and a Gamification Engine for XP, badge, streak, and leaderboard management, the platform delivers a complete and technically sophisticated programming education experience.

All eight test cases passed successfully. Sandpack delivered in-browser code execution at an average of 1.4 seconds with no server infrastructure cost. The Clerk feature gate correctly enforced Pro subscription restrictions in 100 percent of test cases with no database query overhead. Neon Postgres delivered warm query performance under 50 milliseconds and acceptable cold start performance at 780 milliseconds. Stripe webhook processing activated Pro subscriptions within 1.8 seconds of payment completion. Progress persistence was confirmed across sessions with no data loss.

Beyond its technical achievements, Codebox establishes a new design paradigm for programming education platforms: one where the act of writing correct code is rewarded with visible progression through a pixel-art game world, where subscription access is gated behind dungeon doors rather than paywall screens, and where the entire experience is delivered through a serverless architecture that scales to global student populations with minimal infrastructure management. The platform provides both a working product and a reference architecture for the next generation of gamified educational applications.

7.2 Future Scope

7.2.1 AI-Powered Code Hints and Adaptive Difficulty

Future development could integrate an AI hint system that uses a large language model to analyze a student's incorrect code submission and generate a Socratic hint that guides them

toward the correct solution without revealing the answer directly. This would be implemented as a Server Action that sends the student's code, the expected output, and the challenge description to an LLM API, and returns a context-aware hint rendered in the pixel-art chat bubble style of an in-game NPC mentor character. Adaptive difficulty could adjust the XP multiplier and challenge complexity based on the student's historical performance metrics.

7.2.2 Multiplayer Code Battles

A competitive multiplayer mode where two students race to solve the same code challenge in real time would significantly increase the social engagement dimension of the platform. This could be implemented using WebSockets through a service such as Pusher or Ably, where both students see a split-screen battle arena in the pixel-art style, with their opponent's progress indicator and a countdown timer visible in real time. The first student to submit a correct solution wins the battle, earns bonus XP, and advances their battle rank on a separate competitive leaderboard.

7.2.3 Mobile Application

A dedicated React Native mobile application would extend Codebox to the mobile-first student demographic. The Next.js API layer is already mobile-ready as it exposes standard HTTP endpoints and Server Actions accessible from any HTTP client. The Sandpack execution environment is available as a React Native compatible package, enabling in-app code execution on mobile devices. Push notifications through Expo's notification system would support streak reminders and leaderboard challenge alerts.

7.2.4 Instructor Course Builder

A visual course builder interface for instructors would allow educators to create new course worlds, design lesson rooms, configure code challenge templates, specify expected outputs, set XP values, and publish content without requiring direct database access. The builder would be implemented as a protected admin section of the Next.js application using Clerk's organization feature to grant instructor roles. A drag-and-drop lesson sequencer with a pixel-art dungeon layout preview would allow instructors to design the visual world map that students navigate through their course.

7.2.5 Completion Certificates with Blockchain Verification

Issuing verifiable completion certificates upon finishing a course world would add credential value to the Codebox learning experience. Certificates could be minted as non-fungible tokens on a low-cost blockchain such as Polygon, with the certificate metadata stored on IPFS and the student's wallet address recorded in the Neon Postgres database. Students could share a public verification link that allows employers or academic institutions to confirm the certificate's authenticity without trusting any central authority.

REFERENCES

7. React Team (2024). React Documentation - A JavaScript Library for Building User Interfaces. Available at https://react.dev

8. Bhatt, A., & Patel, V. (2022). OSINT Automation for Cyber Investigations. International Journal of Cybersecurity Research, 4(2), 45-58.

9. Sharma, R., & Kumar, P. (2023). Web-Based Threat Intelligence Platforms: A Survey. Journal of Network Security, 8(1), 12-29.

10. ip-api.com (2024). IP Geolocation API Documentation. Available at https://ip-api.com/docs

11. dnspython Developers (2024). dnspython - A DNS Toolkit for Python. Available at https://www.dnspython.org

12. Smicallef, S. (2024). SpiderFoot - Open Source Intelligence Automation. GitHub. Available at https://github.com/smicallef/spiderfoot

13. Next.js Team (2024). Next.js 14 Documentation - App Router. Vercel. Available at https://nextjs.org/docs

14. Neon Technologies (2024). Neon Serverless Postgres Documentation. Available at https://neon.tech/docs

15. Drizzle Team (2024). Drizzle ORM Documentation - TypeScript ORM for SQL. Available at https://orm.drizzle.team

16. Clerk Inc. (2024). Clerk Authentication and User Management Documentation. Available at https://clerk.com/docs

17. CodeSandbox (2024). Sandpack: A Component Toolkit for In-Browser Code Editing. Available at https://sandpack.codesandbox.io

18. Stripe Inc. (2024). Stripe Payments and Subscription API Documentation. Available at https://stripe.com/docs

19. Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From Game Design Elements to Gamefulness: Defining Gamification. Proceedings of the 15th International Academic MindTrek Conference, pages 9 to 15.

20. Hamari, J., Koivisto, J., & Sarsa, H. (2014). Does Gamification Work? A Literature Review of Empirical Studies on Gamification. Hawaii International Conference on System Sciences, pages 3025 to 3034.

21. Subhash, S., & Cudney, E. A. (2018). Gamified Learning in Higher Education: A Systematic Review of the Literature. Computers in Human Behavior, Volume 87, pages 192 to 206.